REAL TIME OPERATING SYSTEM

RTOS LAB

Assignment 1

Name: **Varun B Karmikanda**

USN: 261100610029

Program: ME Computer Science & Engineering

November 4, 2026

1. LED Blink

Question: Blink the onboard LED of the STM32 Blue Pill using bare-metal programming.

Code:

```c
#include <stdint.h>
#include "stm32f103_regs.h"

static volatile uint32_t g_msticks;

void SysTick_Handler(void)
{
    g_msticks++;
}

static void delay_ms(uint32_t ms)
{
    uint32_t target = g_msticks + ms;
    while (g_msticks < target) {
    }
}

int main(void)
{
    RCC->APB2ENR |= RCC_APB2ENR_IOPCEN;

    GPIOC->CRH &= ~(0xFU << 20);
    GPIOC->CRH |= (GPIO_CNF_MODE_OUTPUT_PP_2MHZ << 20);

    SYSTICK->LOAD = (8000000U / 1000U) - 1U;
    SYSTICK->VAL = 0;
    SYSTICK->CTRL = SYSTICK_CTRL_CLKSOURCE |
                    SYSTICK_CTRL_TICKINT |
                    SYSTICK_CTRL_ENABLE;

    for (;;) {
        GPIOC->ODR ^= (1U << 13);
        delay_ms(1000);
    }
}
```

Output: The onboard PC13 LED blinks at a one-second interval.

2. Three Thread Blink

Question: Create three independent FreeRTOS tasks, each blinking its own LED at a fixed interval:

- PC13: 1 second
- PA0: 2 seconds
- PA1: 3 seconds

Code:

```
#include <stdint.h>
#include "stm32f103_regs.h"
#include "FreeRTOS.h"
#include "task.h"

typedef struct {
    GPIO_TypeDef *port;
    uint32_t pin;
    uint32_t period_ms;
} led_task_args_t;

static const led_task_args_t led_a = { GPIOC, 13, 1000 };
static const led_task_args_t led_b = { GPIOA, 0, 2000 };
static const led_task_args_t led_c = { GPIOA, 1, 3000 };
static const led_task_args_t led_d = { GPIOA, 2, 4000 };
static const led_task_args_t led_e = { GPIOA, 3, 5000 };

static void gpio_output_init(GPIO_TypeDef *port, uint32_t pin)
{
    volatile uint32_t *cr = (pin < 8) ? &port->CRL : &port->CRH;
    uint32_t shift = (pin % 8U) * 4U;
    *cr &= ~(0xFU << shift);
    *cr |= (GPIO_CNF_MODE_OUTPUT_PP_2MHZ << shift);
}

static void led_task(void *pvParameters)
{
    const led_task_args_t *args =
        (const led_task_args_t *)pvParameters;
    for (;;) {
        args->port->ODR ^= (1U << args->pin);
        vTaskDelay(pdMS_TO_TICKS(args->period_ms));
    }
}

int main(void)
{
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_IOPCEN;
    gpio_output_init(led_a.port, led_a.pin);
    gpio_output_init(led_b.port, led_b.pin);
    gpio_output_init(led_c.port, led_c.pin);
    gpio_output_init(led_d.port, led_d.pin);
    gpio_output_init(led_e.port, led_e.pin);

    xTaskCreate(led_task, "led_1s", configMINIMAL_STACK_SIZE,
                (void *)&led_a, tskIDLE_PRIORITY + 1, NULL);
    xTaskCreate(led_task, "led_2s", configMINIMAL_STACK_SIZE,
```

```
48                  (void *)&led_b, tskIDLE_PRIORITY + 1, NULL);
49      xTaskCreate(led_task, "led_3s", configMINIMAL_STACK_SIZE,
50                  (void *)&led_c, tskIDLE_PRIORITY + 1, NULL);
51      xTaskCreate(led_task, "led_4s", configMINIMAL_STACK_SIZE,
52                  (void *)&led_d, tskIDLE_PRIORITY + 1, NULL);
53      xTaskCreate(led_task, "led_5s", configMINIMAL_STACK_SIZE,
54                  (void *)&led_e, tskIDLE_PRIORITY + 1, NULL);
55
56      vTaskStartScheduler();
57      for (;;) {
58      }
59  }
60
61  void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName
        )
62  {
63      (void)xTask;
64      (void)pcTaskName;
65      for (;;) {
66      }
67  }
```

Output: The five configured task intervals are observed on the connected LEDs.

3. Three Thread Priority Switch

Question: Create independent LED tasks with different priorities and observe priority-based scheduling.

Code:

```
#include <stdint.h>
#include "stm32f103_regs.h"
#include "FreeRTOS.h"
#include "task.h"

static volatile uint32_t g_msticks;

typedef struct {
    GPIO_TypeDef *port;
    uint32_t pin;
    uint32_t period_ms;
} led_task_args_t;

void vApplicationTickHook(void)
{
    g_msticks++;
}

static const led_task_args_t led_a = { GPIOA, 3, 500 };
static const led_task_args_t led_b = { GPIOA, 0, 450 };
static const led_task_args_t led_c = { GPIOA, 1, 400 };
static const led_task_args_t led_d = { GPIOA, 2, 350 };

static void delay_ms(uint32_t ms)
{
    uint32_t target = g_msticks + ms;
    while (g_msticks < target) {
    }
}

static void gpio_output_init(GPIO_TypeDef *port, uint32_t pin)
{
    volatile uint32_t *cr = (pin < 8) ? &port->CRL : &port->CRH;
    uint32_t shift = (pin % 8U) * 4U;
    *cr &= ~(0xFU << shift);
    *cr |= (GPIO_CNF_MODE_OUTPUT_PP_2MHZ << shift);
}

static void led_task(void *pvParameters)
{
    const led_task_args_t *args =
        (const led_task_args_t *)pvParameters;
    for (;;) {
        args->port->ODR ^= (1U << args->pin);
        delay_ms(args->period_ms);
    }
}

int main(void)
{
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN | RCC_APB2ENR_IOPCEN;
    gpio_output_init(led_a.port, led_a.pin);
```

```
53      gpio_output_init(led_b.port, led_b.pin);
54      gpio_output_init(led_c.port, led_c.pin);
55      gpio_output_init(led_d.port, led_d.pin);
56
57      xTaskCreate(led_task, "led_1s", configMINIMAL_STACK_SIZE,
58                  (void *)&led_a, tskIDLE_PRIORITY + 3, NULL);
59      xTaskCreate(led_task, "led_2s", configMINIMAL_STACK_SIZE,
60                  (void *)&led_b, tskIDLE_PRIORITY + 2, NULL);
61      xTaskCreate(led_task, "led_3s", configMINIMAL_STACK_SIZE,
62                  (void *)&led_c, tskIDLE_PRIORITY + 1, NULL);
63      xTaskCreate(led_task, "led_4s", configMINIMAL_STACK_SIZE,
64                  (void *)&led_d, tskIDLE_PRIORITY + 0, NULL);
65
66      vTaskStartScheduler();
67      for (;;) {
68      }
69  }
70
71  void vApplicationStackOverflowHook(TaskHandle_t xTask, char *pcTaskName
        )
72  {
73      (void)xTask;
74      (void)pcTaskName;
75      for (;;) {
76      }
77  }
78
79  void vApplicationIdleHook(void)
80  {
81      __asm volatile ("wfi");
82  }
```

Output: The higher-priority task executes first whenever multiple tasks are ready.

4. Recursive and Addition Programs: GDB Trace and Debugging

Question: Trace the recursive and addition programs using GDB output and debug them.

4.1 Recursive Program

Code:

```c
int rec_sum(int i)
{
    if (i <= 1)
        return i;
    else
        return i + rec_sum(i - 1);
}

int main(void)
{
    int sum = 0;
    sum = rec_sum(10);
    return 0;
}
```

Output: The calls proceed from `rec_sum(10)` to `rec_sum(1)` and return through the stack. The final value is `sum = 55`.

```
PS C:\Users\VarunKarmikanda\msis\sem1\rtos> wsl
Reading symbols from ./recursion_gdb...
(gdb) break rec_sum
Breakpoint 1 at 0x1138: file recursion.c, line 2.
(gdb) run
Starting program: /mnt/c/Users/VarunKarmikanda/msis/sem1/rtos/assignments/261100610029_RTOS_LAB_C01/04-recursion/recursion_gdb

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, rec_sum (i=10) at recursion.c:2
2           if(i <= 1) return i;
(gdb) print i
$1 = 10
(gdb) info locals
No locals.
(gdb) backtrace
#0  rec_sum (i=10) at recursion.c:2
#1  0x0000555555555174 in main () at recursion.c:8
(gdb) continue
Continuing.

Breakpoint 1, rec_sum (i=9) at recursion.c:2
2           if(i <= 1) return i;
(gdb) print i
$2 = 9
(gdb) backtrace
#0  rec_sum (i=9) at recursion.c:2
#1  0x0000555555555150 in rec_sum (i=10) at recursion.c:3
#2  0x0000555555555174 in main () at recursion.c:8
(gdb) continue
Continuing.

Breakpoint 1, rec_sum (i=8) at recursion.c:2
2           if(i <= 1) return i;
(gdb) continue
Continuing.
```

```
PS C:\Users\VarunKarmikanda\msis\sem1\rtos> wsl

Breakpoint 1, rec_sum (i=7) at recursion.c:2
2           if(i <= 1) return i;
(gdb) continue
Continuing.

Breakpoint 1, rec_sum (i=6) at recursion.c:2
2           if(i <= 1) return i;
(gdb) continue
Continuing.

Breakpoint 1, rec_sum (i=5) at recursion.c:2
2           if(i <= 1) return i;
(gdb) continue
Continuing.

Breakpoint 1, rec_sum (i=4) at recursion.c:2
2           if(i <= 1) return i;
(gdb) continue
Continuing.

Breakpoint 1, rec_sum (i=3) at recursion.c:2
2           if(i <= 1) return i;
(gdb) continue
Continuing.

Breakpoint 1, rec_sum (i=2) at recursion.c:2
2           if(i <= 1) return i;
(gdb) print i
$3 = 2
(gdb) backtrace
#0  rec_sum (i=2) at recursion.c:2
#1  0x0000555555555150 in rec_sum (i=3) at recursion.c:3
#2  0x0000555555555150 in rec_sum (i=4) at recursion.c:3
#3  0x0000555555555150 in rec_sum (i=5) at recursion.c:3
#4  0x0000555555555150 in rec_sum (i=6) at recursion.c:3
#5  0x0000555555555150 in rec_sum (i=7) at recursion.c:3
#6  0x0000555555555150 in rec_sum (i=8) at recursion.c:3
#7  0x0000555555555150 in rec_sum (i=9) at recursion.c:3
#8  0x0000555555555150 in rec_sum (i=10) at recursion.c:3
#9  0x0000555555555174 in main () at recursion.c:8
(gdb) continue
Continuing.
```

```
PS C:\Users\VarunKarmikanda\msis\sem1\rtos> wsl

Breakpoint 1, rec_sum (i=1) at recursion.c:2
2           if(i <= 1) return i;
(gdb) print i
$4 = 1
(gdb) backtrace
#0  rec_sum (i=1) at recursion.c:2
#1  0x0000555555555150 in rec_sum (i=2) at recursion.c:3
#2  0x0000555555555150 in rec_sum (i=3) at recursion.c:3
#3  0x0000555555555150 in rec_sum (i=4) at recursion.c:3
#4  0x0000555555555150 in rec_sum (i=5) at recursion.c:3
#5  0x0000555555555150 in rec_sum (i=6) at recursion.c:3
#6  0x0000555555555150 in rec_sum (i=7) at recursion.c:3
#7  0x0000555555555150 in rec_sum (i=8) at recursion.c:3
#8  0x0000555555555150 in rec_sum (i=9) at recursion.c:3
#9  0x0000555555555150 in rec_sum (i=10) at recursion.c:3
#10 0x0000555555555174 in main () at recursion.c:8
(gdb) finish
Run till exit from #0  rec_sum (i=1) at recursion.c:2
0x0000555555555150 in rec_sum (i=2) at recursion.c:3
3           else return i + rec_sum(i - 1);
Value returned is $5 = 1
(gdb) print $rax
$6 = 1
(gdb) continue
Continuing.
[Inferior 1 (process 6718) exited normally]
(gdb)
```

Trace summary:

$$1,\ 3,\ 6,\ 10,\ 15,\ 21,\ 28,\ 36,\ 45,\ 55$$

The ARM disassembly shows the function prologue, stack-frame allocation, argument storage, recursive branch, and return through register `r0`.

4.2 Addition Program

Code:

```c
int mySum(int a, int b)
{
    return a + b;
}

int main(void)
{
    int i = 5, j = 10, sum = 0;
    sum = mySum(i, j);
    return 0;
}
```

Output: The values $i = 5$ and $j = 10$ are passed in registers `r0` and `r1`. The function returns 15 in `r0`, which is stored in `sum`.

```
PS C:\Users\VarunKarmikanda\msis\sem1\rtos> wsl
(gdb) break mySum
Breakpoint 1 at 0x1137: file sum.c, line 3.
(gdb) run
Starting program: /mnt/c/Users/VarunKarmikanda/msis/sem1/rtos/assignments/261100610029_RTOS_LAB_C01/05-sum/sum_gdb

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, mySum (a=5, b=10) at sum.c:3
3           return a+b;
(gdb) print a
$1 = 5
(gdb) print b
$2 = 10
(gdb) info locals
No locals.
(gdb) backtrace
#0  mySum (a=5, b=10) at sum.c:3
#1  0x0000555555555171 in main () at sum.c:9
(gdb) next
4       }
(gdb) finish
Run till exit from #0  mySum (a=5, b=10) at sum.c:4
0x0000555555555171 in main () at sum.c:9
9           sum =mySum(i,j);
Value returned is $3 = 15
(gdb) print $rax
$4 = 15
```